

INFORME TÉCNICO: **AGENTE RAG DNI VALENCIA**

Asignatura:

Inteligencia Artificial

Alumnos:

Mario Luis Mesa

Joel García Martí

1. Introducción

En esta práctica se ha desarrollado un agente basado en la técnica de *Retrieval-Augmented Generation (RAG)* aplicado al corpus de la asociación DNI Valencia. El objetivo principal es construir un sistema capaz de responder preguntas de manera fundamentada utilizando exclusivamente la información contenida en los documentos proporcionados.

El sistema implementado combina recuperación de información mediante embeddings y generación de lenguaje natural mediante modelos LLM, asegurando que las respuestas estén respaldadas por fuentes reales del corpus.

2. Descripción general del sistema

El sistema sigue una arquitectura de tipo single-agent RAG, en la que un único agente gestiona todo este flujo de procesamiento:

1. Carga del corpus de documentos
2. División en fragmentos (chunking)
3. Generación de embeddings
4. Almacenamiento en base vectorial
5. Recuperación de contexto relevante
6. Generación de respuesta mediante LLM
7. Devolución de respuesta con fuentes

La función principal del sistema es:

```
consultar(pregunta: str, conversation_id: Optional[str] = None) -> dict
```

Esta función devuelve:

- **respuesta**: texto generado
- **fuentes**: documentos utilizados
- **chunks**: fragmentos recuperados
- **metrics**: información de latencia
- **trazas**: detalles del proceso

3. Procesamiento del corpus

3.1 Corpus utilizado

Se han utilizado los 16 documentos proporcionados en el enunciado, sin modificaciones.

El corpus contiene información sobre:

- Programas sociales (COLES, RESIS, desayunos)
- Organización de la asociación
- Preguntas frecuentes

3.2 Estrategia de chunking

Se ha implementado una estrategia híbrida:

- Uso de **RecursiveCharacterTextSplitter**
- Tamaño de chunk: 500 caracteres
- Overlap: 100 caracteres

Además, se ha aplicado un tratamiento especial a estructuras tipo:

Q: ...

A: ...

Evitando que pregunta y respuesta queden separadas en distintos fragmentos.

4. Representación semántica y almacenamiento

4.1 Embeddings

Los documentos se transforman en vectores mediante un modelo de embeddings compatible con Ollama, en concreto **nomic-embed-text**.

4.2 Base de datos vectorial

Se utiliza **ChromaDB** como almacenamiento persistente:

- Persistencia local en **.chroma_dni**
- Almacenamiento de:
 - embeddings
 - texto original
 - metadatos (nombre del archivo)

Esto permite recuperar fragmentos relevantes y citar correctamente las fuentes.

5. Recuperación de información (Retrieval)

El sistema utiliza búsqueda por similitud vectorial:

- Top-K inicial: **5 fragmentos**
- Selección basada en similitud semántica

Limitaciones detectadas

Durante las pruebas, se ha observado que:

- Algunas consultas recuperan documentos parcialmente relevantes
- Casos como:
 - COLES
 - Diferencia entre RESIS y COLESpresentan errores de recuperación

Esto impacta negativamente en métricas como:

- context precision
- context recall

6. Generación de respuestas

6.1 Modelos utilizados

Se han evaluado los siguientes modelos:

- Modelos locales (Ollama):
 - qwen2.5:3b
 - llama3.2:3b
- Modelos remotos (PoliGPT)
 - gwen
 - gemma

6.2 Prompting

Se ha diseñado un prompt con las siguientes características:

- Uso exclusivo del contexto recuperado
- Prohibición explícita de inventar información
- Inclusión de instrucciones para:
 - citar fuentes
 - indicar falta de información

6.3 Control de alucinaciones

El sistema incluye:

- Respuesta estándar cuando no hay información suficiente:

“No tengo esa información en los documentos proporcionados”
- Eliminación de fuentes si la respuesta es negativa

7. Benchmark de modelos

7.1 Metodología

Se ha construido un conjunto de preguntas representativas sobre el dominio DNI.

Se han evaluado 4 modelos usando las mismas preguntas y las mismas condiciones.

Métricas recogidas:

- Latencia total
- Calidad subjetiva
- Tokens generados

7.2 Resultados

Los experimentos del benchmark se almacenaron en dos ficheros:

- **benchmark.json**: resultados completos en formato estructurado (pregunta, modelo, respuesta, fuentes y métricas).
- **benchmark.md**: versión en tabla para comparación más cómoda.

En términos generales, los sistemas RAG funcionaron bien cuando el retrieval recuperaba correctamente la información relevante. En esos casos, los modelos respondían de forma adecuada y coherente.

A nivel de comportamiento por tipo de sistema:

- **PoliGPT**: obtuvo respuestas más completas y mejor redactadas, especialmente cuando había que combinar información de varios documentos.
- **Ollama local**: permitió ejecución completamente local, con resultados correctos pero más simples en algunos casos.

Entre los modelos locales:

- **qwen2.5:3b** destacó como el más equilibrado en calidad y velocidad.
- **llama3.2:3b** tuvo un rendimiento similar, ligeramente inferior en velocidad.
- En general, ambos funcionaron bien en preguntas factuales y supieron abstenerse cuando no había información.

Un problema importante apareció en preguntas complejas (por ejemplo, comparaciones tipo RESIS vs COLES), donde el fallo no venía tanto del modelo, sino del retrieval. Cuando los chunks recuperados no eran adecuados, incluso modelos buenos no podían responder correctamente. Esto se reflejó en métricas como *context_precision* y *context_recall*.

7.3 Análisis

Del análisis de métricas se observa lo siguiente:

- **Mejor calidad global:** `qwen` en `PoliGPT`
 - Fue el más preciso en general con una media de 0.917.
 - Falló principalmente en la comparación RESIS vs COLES.
 - A cambio, tuvo la mayor latencia (~10.5s).
- **Modelo más rápido:** `gemma` en `PoliGPT`
 - Latencia media de 3.085s
 - Calidad media de 0.833.
 - Respuestas más rápidas, pero demasiado cortas o menos precisas.
- **Modelos locales** (`qwen2.5:3b` y `llama3.2:3b`):
 - Calidad media aproximada de 0.75.
 - Latencia media alrededor de 5s.
 - Buen comportamiento en preguntas simples y fuera de dominio.
 - Limitaciones en tareas de síntesis o razonamiento más complejo.
- **Rendimiento de tokens:**
 - `qwen2.5:3b`: ~111 tokens/s
 - `llama3.2:3b`: ~107 tokens/s

Conclusión del análisis

El mejor equilibrio global lo ofrece `qwen` en `PoliGPT` (calidad alta, pero más lento).

El más rápido es `gemma` en `PoliGPT`, aunque con menor calidad.

Sin embargo, para el sistema final se mantiene `qwen2.5:3b` en local, porque permite:

- ejecución offline,
- rendimiento aceptable,
- y autonomía del sistema.

La conclusión clave es que el mayor cuello de botella no está solo en el LLM, sino en el sistema de recuperación, especialmente cuando hay que combinar información de varios documentos.

8. Evaluación con RAGAs

Se han utilizado las métricas estándar de RAGAs:

- **Faithfulness:** 0.73
- **Answer relevancy:** 0.48
- **Context precision:** 0.35
- **Context recall:** 0.33

8.1 Interpretación

- Faithfulness relativamente alta → el modelo no inventa demasiado
- Relevancy media → respuestas correctas pero incompletas
- Precision y recall bajos → problemas en recuperación de contexto

Esto confirma que el principal cuello de botella del sistema es el retrieval.

9. Métricas propias

Se han definido dos métricas adicionales:

9.1 Calidad subjetiva

Evalúa manualmente si la respuesta es útil.

Limitación:

- Evaluación simplificada en código

9.2 Uso de fuentes

Evalúa si la respuesta incluye referencias reales del corpus.

Justificación:

- Garantiza trazabilidad
- Refuerza confianza en el sistema

10. Limitaciones del sistema

Las principales limitaciones identificadas son:

- Retrieval imperfecto en consultas complejas
- Dependencia del modelo LLM
- Baja cobertura en preguntas abstractas
- Métricas automáticas limitadas

11. Conclusiones

El sistema desarrollado cumple con los requisitos principales de un agente RAG completo, integra la carga del corpus, el chunking, la generación de embeddings, la recuperación de información, la llamada al modelo de lenguaje y la devolución de respuestas con fuentes. De esta forma, el agente no responde únicamente con el conocimiento interno del LLM, sino que basa sus respuestas en los documentos oficiales de DNI.

Durante las pruebas se ha comprobado que el sistema funciona mejor en preguntas directas, donde la información aparece claramente en el corpus. En cambio, en preguntas más complejas o de síntesis, como las que requieren combinar información de varios documentos, el resultado depende mucho de que se recuperen los chunks adecuados.

El benchmark permitió comparar distintos modelos manteniendo el mismo pipeline. Aunque los modelos de PoliGPT ofrecieron respuestas más completas en algunos casos, se eligió `qwen2.5:3b` mediante Ollama local como modelo principal por su equilibrio entre calidad, velocidad y facilidad de ejecución, además de cumplir el requisito de funcionamiento en local.

La evaluación con RAGAs y las métricas propias confirmó que el sistema es útil, pero también mostró que la principal limitación está en la recuperación de contexto. Por tanto, las mejoras futuras deberían centrarse en optimizar el retrieval, por ejemplo ajustando el tamaño de los chunks, probando otros valores de `top_k`, aplicando arquitectura multi-agente o añadiendo una fase de re-ranking.

En conclusión, el resultado final es un agente RAG funcional y evaluado, capaz de responder preguntas sobre DNI citando las fuentes utilizadas. Aunque todavía tiene margen de mejora, especialmente en la recuperación de información, cumple los objetivos principales de la práctica.